



Réseau

Systemes Embarqués Communicants

TP : Chat sur socket Unix

Auteur :

M. Guillaume PIRAUBE

M. Aymeric ZIMINSKI

Version du
May 22, 2026

Contents

1	Introduction	2
2	Partie TCP	3
2.1	Étape 2 — Connexion au serveur	3
2.2	Étape 3 — Réception de messages du chat général	5
2.3	Étape 4 — Envoi de messages sur le chat général	6
2.4	Étape 5 — Déconnexion du serveur	7
2.5	Étape 6 — Création d'une socket d'écoute UDP	8
2.6	Étape 7 — Détection de la commande de message privé	9
2.7	Étape 8 — Envoi d'un message privé en UDP	10
2.8	Étape 9 — Création d'un thread supplémentaire	12
2.9	Étape 10 — Réception des messages privés	13
3	Conclusion	15

1 Introduction

L'objectif de ce TP est de réaliser un chat box sur socket Unix en utilisant le protocole TCP dans un premier temps pour recevoir et envoyer des messages visibles sur le serveur.

Dans un second temps nous allons utiliser le protocole UDP afin de pouvoir envoyer des messages privés. Le tout en utilisant des thread afin de pouvoir gérer la réception de message publique et privé et l'envoi de message.

2 Partie TCP

2.1 Étape 2 — Connexion au serveur

L'objectif de cette étape est d'établir une connexion TCP avec le serveur central du chat, situé à l'adresse 169.254.61.46 et qui écoute sur le port 4242. La connexion doit être établie *avant* la boucle principale, afin que les threads puissent ensuite utiliser le descripteur de socket pour envoyer et recevoir les messages.

Création de la socket TCP

```
int sd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
if(sd < 0){
    perror("socket");
    return 1;
}
printf("[CONSOLE] Socket TCP cree\n");
```

L'appel système `socket()` crée un point de communication et renvoie un descripteur de fichier (entier) qui sera utilisé par tous les appels réseau suivants. Les trois arguments sont :

- `AF_INET` : domaine de communication **IPv4**.
- `SOCK_STREAM` : type de socket.
- `IPPROTO_TCP` : on impose le protocole **TCP**.

Construction de l'adresse du serveur

```
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(4242);
addr.sin_addr.s_addr = inet_addr("169.254.61.46");
```

Établissement de la connexion

```
int connect_err = connect(sd, (struct sockaddr*)&addr, sizeof(addr));
if(connect_err < 0){
    perror("connect");
}
```

```

    return 1;
}
printf("[CONSOLE] Socket connecte!\n");

```

L'appel `connect()` déclenche le **three-way handshake** TCP : le client envoie un segment SYN, le serveur répond par SYN-ACK, puis le client confirme avec ACK. À l'issue de ces trois paquets, la connexion est établie et le serveur envoie immédiatement son message de bienvenue.

C'est exactement l'échange que on a pu observer sur Wireshark (figure 1).

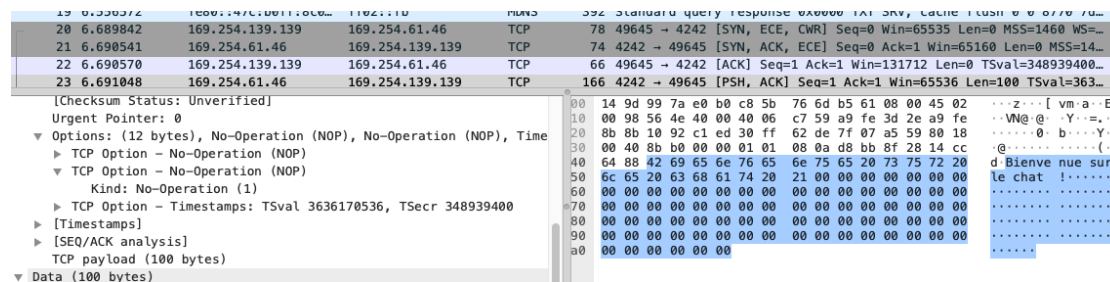


Figure 1: Connexion avec le serveur TCP suivi du message de bienvenue.

2.2 Étape 3 — Réception de messages du chat général

Le serveur fonctionne en mode *broadcast* : dès qu'un client lui transmet un message, il le rediffuse à tous les clients connectés. Notre tâche consiste à lire en continu ce flux entrant dans la boucle principale. Comme suggéré par l'indice du sujet, c'est le **thread enfant** (celui pour lequel `pid[0] == 0` et `pid[1] == 0`) qui se charge de cette lecture, pour ne pas bloquer la saisie clavier gérée par le parent.

```
if(pid[0] == 0 && pid[1] == 0) { // thread enfant
    ssize_t lrecv = recv(sd, &buffer, MSG_LEN, 0);
    if(lrecv == 0){
        printf("Client deconnecte\n");
    } else if(lrecv < 0){
        perror("recv");
    } else {
        buffer[lrecv] = '\0';
        printf("%s \n", buffer);
    }
}
```

L'appel `recv(sd, buffer, MSG_LEN, 0)` lit les données disponibles sur la socket TCP `sd`. C'est un appel **bloquant** par défaut : tant qu'aucun octet n'arrive, le thread reste en attente.

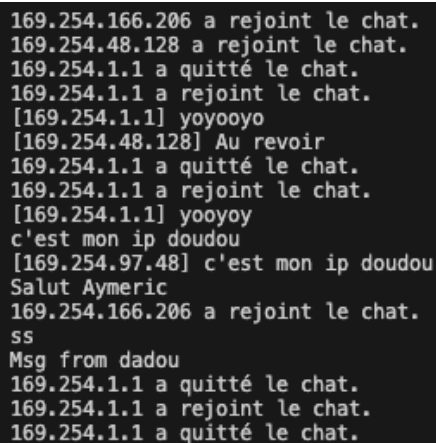
2.3 Étape 4 — Envoi de messages sur le chat général

Il faut pouvoir saisir un message au clavier et l'envoyer au serveur. Cette tâche est confiée au **thread parent** (`pid[0] != 0` et `pid[1] != 0`), de sorte à ce que la lecture clavier bloquante n'empêche pas de recevoir les messages.

```
if(pid[1] != 0 && pid[0] != 0){ // thread parent
    /* Lit le contenu */
    scanf(IN_FILT, buffer);

    else {
        /* Pas une commande */
        ssize_t send_err = send(sd, &buffer, sizeof(buffer), 0);
        if(send_err < 0){
            perror("send");
            return 1;
        }
    }
}
```

La commande `send(sd, buffer, sizeof(buffer), 0)` transmet le `buffer` sur la socket TCP. Le quatrième paramètre (`flags`) vaut 0 pour avoir le comportement par défaut.



```
169.254.166.206 a rejoint le chat.
169.254.48.128 a rejoint le chat.
169.254.1.1 a quitté le chat.
169.254.1.1 a rejoint le chat.
[169.254.1.1] yoyooyo
[169.254.48.128] Au revoir
169.254.1.1 a quitté le chat.
169.254.1.1 a rejoint le chat.
[169.254.1.1] yooyoy
c'est mon ip doudou
[169.254.97.48] c'est mon ip doudou
Salut Aymeric
169.254.166.206 a rejoint le chat.
ss
Msg from dadou
169.254.1.1 a quitté le chat.
169.254.1.1 a rejoint le chat.
169.254.1.1 a quitté le chat.
```

Figure 2: Envois et reception du message sur le chat général

2.4 Étape 5 — Déconnexion du serveur

L'utilisateur signale son souhait de quitter en tapant la commande /q. Le client doit alors fermer proprement la connexion TCP et libérer toutes les ressources réseau avant de se terminer.

```
if(strcmp(buffer, CMD_QUIT) == 0) {  
    /* Commande pour quitter */  
    quit = 1;  
    printf("Au revoir !\n");  
    int shutdown_err = shutdown(sd, SHUT_RDWR);  
    close(sd);  
    close(sd_UDP_in);  
}
```

La séquence employée combine `shutdown()` et `close()`. Cette distinction est importante :

- `shutdown(sd, SHUT_RDWR)` envoie un segment TCP FIN au serveur et interdit toute lecture/écriture future sur la socket. Cela permet au serveur d'être prévenu **immédiatement** de la fermeture.
- `close(sd)` libère ensuite le descripteur côté noyau.

La variable `quit` est locale à chaque thread (`fork()` duplique l'espace mémoire). Mettre `quit = 1` dans le parent n'arrête *pas* les enfants.

```
if(pid != 0) { // thread parent  
    /* Termine le thread enfant */  
    kill(pid, SIGTERM);  
}  
return 0;
```


2.5 Étape 6 — Création d'une socket d'écoute UDP

Nous souhaitons maintenant pouvoir envoyer et recevoir des messages privés. Pour se faire, nous commençons par réaliser le socket d'écoute UDP sur le port 5555.

```
int sd_UDP_in = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if(sd_UDP_in < 0){
    perror("socket");
    return 1;
}
printf("[CONSOLE] Socket UDP cree\n");

struct sockaddr_in addr_UDP;
addr_UDP.sin_family = AF_INET;
addr_UDP.sin_port = htons(5555);
addr_UDP.sin_addr.s_addr = INADDR_ANY;

int err_bind_UDP = bind(sd_UDP_in, (struct sockaddr*)&addr_UDP, sizeof(addr_UDP));
if(err_bind_UDP < 0){
    perror("bind");
    return 1;
}
printf("[CONSOLE] Bind UDP passe\n");
```

Plusieurs différences notables par rapport à l'étape TCP :

- SOCK_DGRAM (au lieu de SOCK_STREAM) : on demande un mode **datagramme**. Chaque envoi est un message indépendant, sans connexion préalable, sans garantie d'ordre ni de livraison.
- IPPROTO_UDP : protocole UDP.
- INADDR_ANY (valeur 0.0.0.0) : on attache la socket à **toutes les interfaces réseau** de la machine. Le client acceptera donc des datagrammes arrivant aussi bien par l'interface filaire que par le loopback (127.0.0.1), ce qui sera utile pour tester localement à l'étape 10.
- bind() associe la socket au port 5555.

2.6 Étape 7 — Détection de la commande de message privé

Nous allons maintenant implémenter le fait de pouvoir envoyer des messages privés via la commande suivante `/mp [IP] [MESSAGE]`, par exemple :

```
/mp 169.254.25.1 Ceci est un message privé
```

Le fichier `parse_command.h` fourni possède la fonction `parse_mp` qui permet d'extraire l'adresse IP ainsi que le message. On l'appelle donc dans la même section que celle de la commande `quit` :

```
char ip_addr_mp[50];
char msg_MP[MSG_LEN];
int etat_mp;

etat_mp = parse_mp(buffer, ip_addr_mp, msg_MP);

if(etat_mp == -1){
    printf("Erreur : %s", msg_MP);
}
else if(etat_mp == 1){
    /* Traitement de l'envoi UDP -- voir etape 8 */
}
else{
    /* etat_mp == 0 : ce n'est pas un /mp, on envoie en broadcast TCP */
    ssize_t send_err = send(sd, &buffer, sizeof(buffer), 0);
    /* ... */
}
```

Le contrat de retour est synthétisé ci-dessous :

Valeur	Signification
-1	Erreur (compilation regex échouée)
0	La saisie n'est pas une commande <code>/mp</code>
1	La commande <code>/mp</code> a été reconnue et les champs extraits

2.7 Étape 8 — Envoi d'un message privé en UDP

Une fois la commande /mp implémentée, il faut transmettre le message directement à l'IP cible sur son port 5555, sans passer par le serveur. Comme indiqué par l'indice, on crée un **socket UDP temporaire** dédiée à l'envoi (la socket sd_UDP_in de l'étape 6 est réservée à l'écoute).

```
else if(etat_mp == 1){
    // On envoie le MP
    int sd_UDP_out = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    struct sockaddr_in addr_UDP_out;
    addr_UDP_out.sin_family = AF_INET;
    addr_UDP_out.sin_port = htons(5555);
    addr_UDP_out.sin_addr.s_addr = inet_addr(ip_addr_mp);

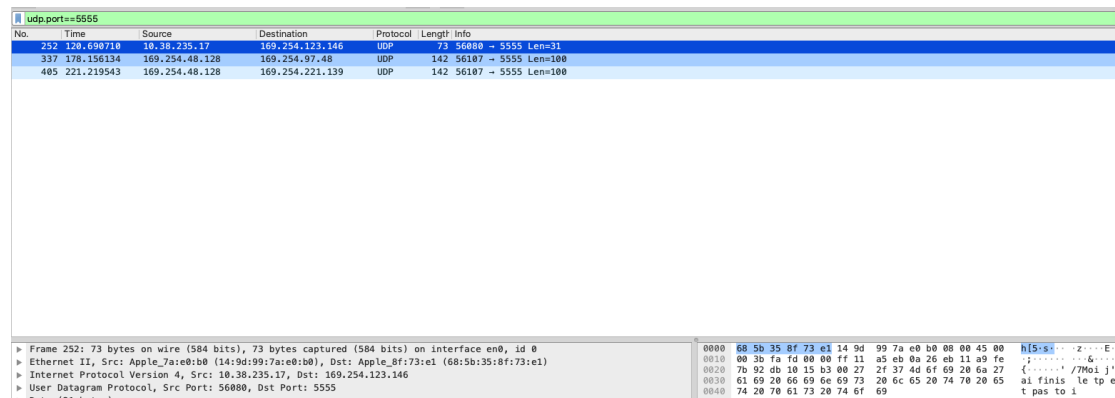
    int bytes_sent = sendto(sd_UDP_out, msg_MP, strlen(msg_MP), 0,
                           (struct sockaddr*)&addr_UDP_out,
                           sizeof(addr_UDP_out));

    if(bytes_sent < 0){
        perror("sendto");
    }
    close(sd_UDP_out);
    printf("MP envoye : %s", msg_MP);
}
```

L'appel système clé ici est `sendto()`, qui diffère de `send()` utilisé en TCP :

- `send()` suppose un socket déjà connectée : la destination est implicite.
- `sendto()` prend **explicitement** l'adresse de destination dans ses deux derniers arguments. C'est indispensable en UDP puisqu'aucune connexion préalable n'a été établie : chaque datagramme transporte sa propre adresse cible.

À noter que l'on envoie `strlen(msg_MP)` octets et non `sizeof(buffer)` : on ne transmet que le contenu réellement utile, sans le padding du buffer (figure 3).



No.	Time	Source	Destination	Protocol	Length	Info
252	120.690710	10.38.235.17	169.254.123.146	UDP	73	56800 → 5555 Len=31
337	178.156134	169.254.48.128	169.254.97.48	UDP	142	56107 → 5555 Len=100
405	221.219543	169.254.48.128	169.254.221.139	UDP	142	56107 → 5555 Len=100

Frame 252: 73 bytes on wire (584 bits), 73 bytes captured (584 bits) on interface en0, id 0	0000	68 5b 35 8f 73 e1 14 9d 99 7a e0 b0 00 00 45 00	h[5:s] ... 2...E:
► Ethernet II, Src: Apple_7a:e0:b0 (14:9d:99:7a:e0:b0), Dst: Apple_8f:73:e1 (68:5b:35:8f:73:e1)	0010	00 3b fa fd 00 00 ff 11 a5 eb 0a 26 eb 11 a9 fe	 &... ..
► Internet Protocol Version 4, Src: 10.38.235.17, Dst: 169.254.123.146	0020	7b 82 db 10 15 b3 00 2f 37 4d 6f 69 20 6a 27	 /7Moi j'
► User Datagram Protocol, Src Port: 56800, Dst Port: 5555	0030	61 69 20 66 69 6e 69 73 20 6c 65 20 74 70 20 65	ai finis le tp e
	0040	74 20 70 61 73 20 74 6f 69	t pas to i

Figure 3: Capture Wireshark montrant l'envoi d'un message privé

2.8 Étape 9 — Création d'un thread supplémentaire

Le client doit maintenant accomplir **trois** tâches concurrentes : lire le clavier, recevoir les messages broadcast TCP, et recevoir les messages privés UDP. Avec un seul thread parent et un seul enfant, on ne pouvait gérer que deux flots bloquants. On ajoute donc un second `fork()`, lui aussi exécuté par le parent.

```
int pid[2] = {0, 0};
pid[0] = fork();
if(pid[0] != 0) { // thread parent
    pid[1] = fork();
}
```

Analyse des valeurs de `pid[0]` et `pid[1]` dans chaque processus

On peut ainsi synthétiser les processus dans le tableau suivant :

Processus	pid[0]	pid[1]	Rôle dans le client
Parent	PID du 1 ^{er} enfant	PID du 2 nd enfant	Lecture du prompt + envoi (TCP/UDP)
1 ^{er} enfant	0	0	Réception TCP (chat broadcast)
2 nd enfant	PID hérité ($\neq 0$)	0	Réception UDP (messages privés)

Il faut ensuite tuer les deux enfants avant la fin du programme, sinon ils deviendraient orphelins et continueraient à écouter sur les sockets :

```
if(pid[1] != 0) { // thread parent
    kill(pid[1], SIGTERM);
    kill(pid[0], SIGTERM);
}
```

2.9 Étape 10 — Réception des messages privés

Le second enfant (identifié par `pid[1] == 0 && pid[0] != 0`) est dédié à la scrutation de la socket UDP `sd_UDP_in` attachée au port 5555 à l'étape 6.

```
if(pid[1] == 0 && pid[0] != 0){
    // Second enfant : reception UDP
    socklen_t addrlen = sizeof(addr_UDP);
    ssize_t recsize = recvfrom(sd_UDP_in, (void*)buffer, sizeof(buffer), 0,
                              (struct sockaddr*)&addr_UDP, &addrlen);

    if(recsize == 0){
        printf("Client deconnecte\n");
    } else if(recsize < 0){
        perror("recv");
    } else {
        buffer[recsize] = '\0';
        printf("%s \n", buffer);
    }
}
```

L'appel central est `recvfrom()`, le pendant UDP de `recv()`.

Protocole de test local

Pour valider la chaîne complète sans dépendre d'un autre poste, nous avons suivis les étapes suivantes :

1. Faire tourner le premier client `./client` sur le port 5555.
2. Modifier la valeur du port d'écoute (par exemple `htons(5556)`), recompiler, et lancer un deuxième client en parallèle.
3. Depuis le deuxième client, saisir `/mp 127.0.0.1 Test local` pour s'adresser au deuxième client.
4. Vérifier que le premier client affiche bien le message reçu.

3 Conclusion

Au cours de ce TP, nous avons réalisé un client de chat combinant les protocoles TCP et UDP. Nous avons pu réaliser la première partie sur le protocole TCP, qui permet d'envoyer et de recevoir les messages globaux sur le serveur. Nous avons pu également réaliser la deuxième partie avec le protocole UDP pour la gestion des messages privées sans passer par le serveur en répartissant le tout sur trois processus parallèles pour gérer simultanément la saisie clavier, la réception des messages publics et celle des messages privés.